StyleSync

Smart wardrobe, explainable outfit recommender and IoT hanger tags

Complete Project Documentation

Front end, back end, database, IoT hardware, firmware, security and testing, written so that a report can be made from it without opening the code.

B.Tech Information Technology capstone project
Project owner: Parth Mhatre
Document status: matches the finished code as of October 2026

Contents

How to use this document 5

1. Project overview 6

1.1 The problem . . . 6
1.2 The idea . . . 6
1.3 Objectives . . . 6
1.4 Who uses it . . . 6
1.5 What was built (summary) . . . 7
1.6 Size of the project . . . 7

2. System architecture 8

2.1 The big picture . . . 8
2.2 Layers . . . 8
2.3 Two kinds of callers . . . 9
2.4 Request flow of an ordinary page . . . 9
2.5 Folder structure . . . 9

3. Technology stack and components 10

3.1 Software: back end . . . 10
3.2 Software: front end . . . 10
3.3 Cloud services . . . 11
3.4 Hardware components . . . 11
3.5 Development and test tools . . . 11

4. IoT hanger-tag system 12

4.1 What it does . . . 12
4.2 Wiring . . . 12
4.3 The firmware (rfid_scanner.ino) . . . 12
4.4 The scan, step by step . . . 14
4.5 The rules inside process_scan . . . 15
4.6 Pairing a reader and linking tags . . . 15
4.7 Tags belong to the account, not to the card . . . 15
4.8 Notifications . . . 16
4.9 Working without hardware . . . 16
4.10 Screenshots of the finished IoT feature . . . 17

5. Back-end implementation 19

5.1 Module by module . . . 19
5.2 Pages and API routes . . . 20

5.3 Authentication and sessions . . . 20
5.4 Photo handling . . . 21
6. The outfit recommendation algorithm 22
6.1 Pipeline . . . 22
6.2 Inputs . . . 22
6.3 Points for one piece . . . 22
6.4 Occasion comes first . . . 23
6.5 Scoring a whole outfit . . . 23
6.6 Choosing the look . . . 23
6.7 The explanation . . . 23
7. Database design 25
7.1 Security of the data . . . 25
7.2 SQL files (run once, in this order, in the Supabase SQL Editor) . . . 26
8. Front-end design 27
8.1 Design system . . . 27
8.2 Pages . . . 27
8.3 Shared scripts . . . 28
8.4 Principles that were followed . . . 28
8.5 Screenshots . . . 28
9. Security 34
9.1 Known limits (honest list for the report) . . . 34
10. Testing and results 36
10.1 Automated checks: 126 passed, 0 failed . . . 36
10.2 Browser checks (headless Chrome, 24 checks in total) . . . 36
10.3 Real hardware tests (done by the project owner) . . . 36
11. Installation, running and demonstration 38
11.1 First-time setup . . . 38
11.2 Setting up the reader . . . 38
11.3 College demonstration with a phone hotspot . . . 38
11.4 Three-minute demonstration script . . . 38
12. Problems faced and how they were solved 40
13. Limitations and future work 41
13.1 Limitations . . . 41
13.2 Future work . . . 41

14. Appendix A: file inventory **42**

Environment variables (.env) 43

15. Appendix B: glossary **44**

How to use this document

This PDF is the single source of truth for the StyleSync project. Everything that was built is described here: what it is, which components and libraries were used, how the parts are connected, how the data moves, and how it was tested. The person writing the college report can take each chapter below and turn it into the matching report chapter.

Report chapter (suggested)	Take the material from
Introduction, problem statement, objectives	Chapter 1
System design and architecture	Chapters 2 and 3
Hardware and IoT design	Chapter 4
Back-end implementation	Chapter 5
Outfit recommendation algorithm	Chapter 6
Database design	Chapter 7
Front-end design and user interface	Chapter 8 (screenshots included)
Security	Chapter 9
Testing and results	Chapter 10
Installation and demonstration	Chapter 11
Problems faced and solutions	Chapter 12
Limitations and future work	Chapter 13
Appendices (files, environment, commands, glossary)	Chapters 14 and 15

TIP

Not in this document and still to be written by the report author: the literature survey (existing wardrobe and outfit apps such as Stylebook, Cladwell, Acloset), the introduction paragraph about the motivation in his own words, the conclusion, and photographs of the real hardware.

1. Project overview

1.1 The problem

People own many clothes but wear a small part of them. They forget what they own, repeat the same outfit within a week, and spend time deciding what to wear. Existing wardrobe apps need manual typing for every outfit and have no link to the physical wardrobe.

1.2 The idea

StyleSync is a website where a user stores a digital copy of their wardrobe (photo, category, colour, season, occasion, style). A rule-based **Outfit Matcher** builds complete outfits and explains every choice in plain sentences. An **IoT hanger-tag system** connects the real wardrobe to the website: an RFID tag on each hanger, and an ESP32 reader that tells the website when a garment is taken off the rail. The site then logs the piece as worn, warns about repeats within a week, and notifies the user on the website bell and as a real browser pop-up. The reader itself answers with beep patterns.

1.3 Objectives

- Let a user keep a clean digital wardrobe with photos, filters and search.
- Suggest explainable outfits (no black-box AI): every suggestion lists its reasons.
- Apply a one-week no-repeat rule to pieces and to whole outfits.
- Log what is worn automatically using RFID hanger tags and an ESP32 reader, with no manual typing.
- Notify the user on the website and in the browser, even when the site is not the active tab.
- Be secure by design: validation on the server, rate limiting, hashed passwords, generic errors, per-user data isolation.
- Be simple to explain: lean, commented code that a student can read.

1.4 Who uses it

A single user managing their own wardrobe (each account is completely separate), and the examiners who evaluate the project's functional completeness, code quality and hardware integration.

1.5 What was built (summary)

Area	What exists
Accounts	Sign up, log in, log out, sessions with automatic logout after 2 hours
Wardrobe	Add pieces with photo and details (with an "Other" option for colour and similar), gallery, category filters, search, delete, hanger-tag status, wear count and last-worn time
Outfit Matcher	Choose occasion, season, optional style and colour mood; get a complete look with a match percentage and reasons; Shuffle for a new look; Save look; Wear this today
History	Worn looks, saved looks, hanger looks (pieces taken off the rail together), remove from saved
Profile	Favourite styles and colours that gently influence the matcher
Dashboard	Greeting, statistics cards that link to pages, quick start for the matcher
IoT	Pair readers, link tags to pieces, scan logging, repeat warnings, beeps, live Hangers page, simulate scan without hardware
Notifications	Bell with unread count, real browser push (Web Push), in-page pop-ups with close button, 60-day "you have not worn this" reminder
Security	See chapter 9
Quality	126 automated checks plus browser checks and real-hardware tests

1.6 Size of the project

Part	Approximate size
Python back end (14 modules)	about 1,600 lines
HTML templates (9 page files, 3 shared parts)	about 700 lines
JavaScript (13 files)	about 1,150 lines
CSS (one theme file)	about 1,300 lines
Arduino firmware (1 sketch)	about 200 lines
SQL (4 files) and tests (126 checks)	four SQL files, one test script and a fake database

2. System architecture

2.1 The big picture

StyleSync has five cooperating parts. The browser shows the user interface. The Flask server holds all the logic and is the only part allowed to talk to the database. Supabase stores data, handles login and keeps photos. The ESP32 reader reports scans over Wi-Fi. A push service (run by the browser maker) delivers notification pop-ups.

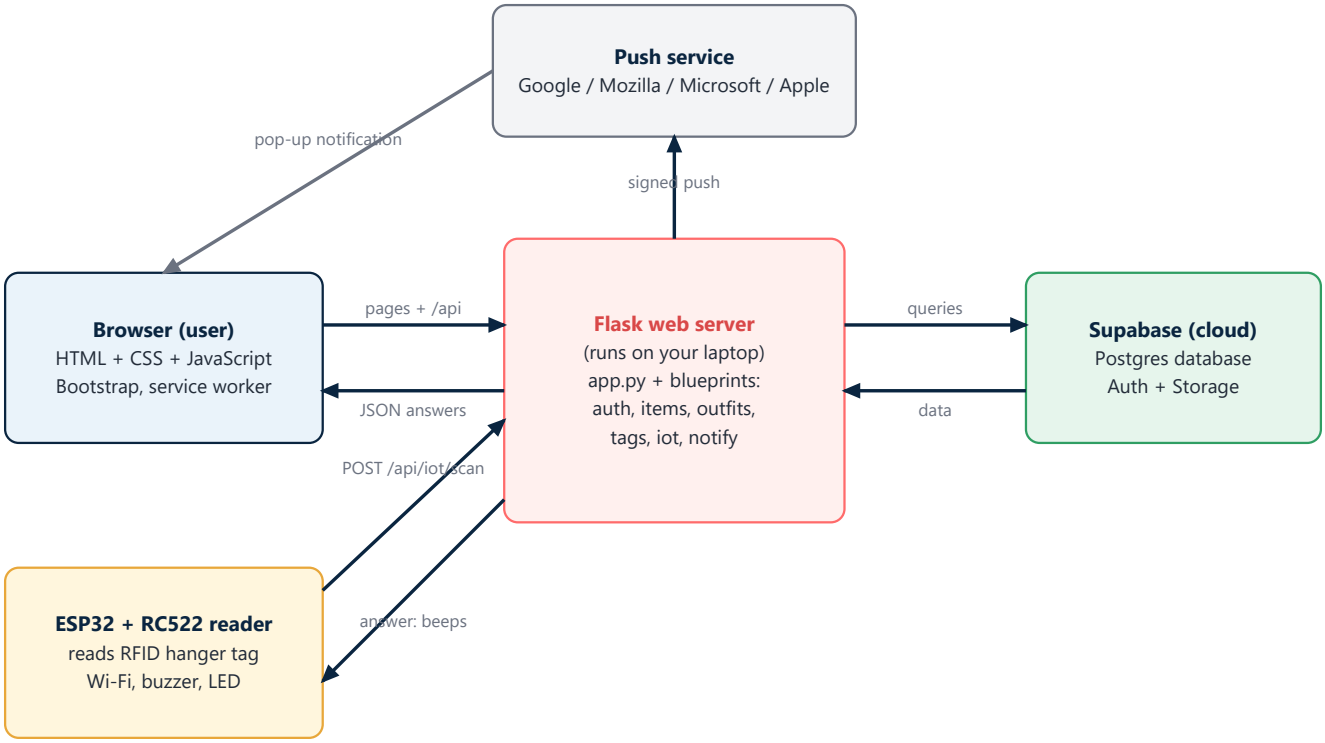


Figure 1. How the five parts are connected. The browser never talks to Supabase directly.

2.2 Layers

Layer	Technology	Responsibility
Presentation	HTML (Jinja templates), CSS, JavaScript, Bootstrap 5	Show pages, collect input, call the API, draw outfit cards, toasts and the bell
Application (back end)	Python Flask with blueprints	Authentication, validation, outfit logic, scan logic, notifications, security headers
Data	Supabase (PostgreSQL, Auth, Storage)	Store users, clothes, looks, tags, readers, notifications, photos
Device	ESP32 + RC522 RFID + buzzer (Arduino C++)	Read the tag id and send it to the server; make beeps from the answer
Delivery	Web Push (VAPID), service worker	Show pop-ups even if the website is not the open tab

2.3 Two kinds of callers

The server has two different kinds of clients, and they are protected differently. A **person in a browser** logs in, gets a signed cookie and sends a CSRF token with every change. A **reader** (ESP32) cannot log in and has no cookie. It proves who it is with a secret **device key** sent in a header. That key can do exactly one thing: report scans for the account that paired it.

2.4 Request flow of an ordinary page

1. The browser asks for a page, for example /wardrobe.
2. Flask checks the login cookie (`@login_required`). No cookie: redirect to the landing page.
3. The route reads the user's own data from Supabase (every query filters by the user's id).
4. Jinja fills the template and Flask adds the security headers.
5. The browser loads the CSS and JavaScript. Scripts then call /api routes (with the CSRF token) for actions such as suggest an outfit or add a piece.

2.5 Folder structure

```
StyleSync_Prototype/
  README.md  .env  .env.example  requirements.txt
  app.py  auth.py  validators.py  extensions.py  config.py  db.py     (core, login, security)
  items.py  images.py                                                 (wardrobe and photos)
  outfit_engine.py  outfits.py  options.py                            (matcher, history, lists)
  tags.py  iot.py  notify.py                                          (hanger tags, ESP32, alerts)
  templates/   static/{css,js,images}   database/   firmware/   tools/   tests/   docs/
```

3. Technology stack and components

3.1 Software: back end

Component	Version	Why it is used
Python	3.14 (3.11+ works)	Main language of the project
Flask	3.1.3	Small web framework: routes, templates, sessions. Blueprints split the code per feature
Werkzeug / Jinja2 / MarkupSafe / itsdangerous	3.1.9 / 3.1.6 / 3.0.3 / 2.2.0	Flask's own building blocks: HTTP handling, HTML templates with auto-escaping, signed cookies
Flask-WTF	1.3.0	CSRF protection for every form and API change
Flask-Limiter	4.1.1	Rate limiting (stops password guessing and flooding)
supabase (python client)	2.31.0	Talks to the Supabase database, auth and storage from the server
python-dotenv	1.2.4	Loads secrets from the .env file
Pillow	12.3.0	Re-encodes uploaded photos as clean JPEGs (removes hidden data), resizes them
pywebpush, py-vapid, http_ece, cryptography, requests	2.5.0, 1.9.4, 1.2.1, 50.0.1, 2.34.2	Send real browser push notifications (signed with VAPID keys, encrypted)

All versions are pinned with == in requirements.txt so that a surprise update cannot break or weaken the app.

3.2 Software: front end

Component	Version	Why it is used
HTML + Jinja templates	-	Pages are rendered on the server, so content is escaped automatically
CSS (own theme file, style.css)	-	One file with design tokens (colours, spacing, shadows) so every page looks the same
Bootstrap	5.3.8 (CDN with integrity hash)	Grid, modals, basic components
Bootstrap Icons	1.13.1 (CDN with integrity hash)	Icons
Google Fonts: Poppins and Inter	-	Headings and body text
Plain JavaScript	-	No framework: small files, easy to explain. fetch() calls the API
Service worker (sw.js)	-	Receives push messages and shows the pop-up

3.3 Cloud services

Service	What it provides here
Supabase Postgres	All tables (users' clothes, looks, tags, devices, notifications)
Supabase Auth	Accounts and password hashing (bcrypt). The app never sees a password after sending it
Supabase Storage	Bucket `garment-images` with the cleaned photos (random file names)
Browser push services	Google (Chrome), Mozilla (Firefox), Microsoft (Edge), Apple (Safari). They only deliver the pop-up

3.4 Hardware components

Component	Details	Role
ESP32 DevKit (V1 or similar)	Wi-Fi 2.4 GHz only, 3.3 V logic	The brain of the reader: reads the RFID chip over SPI, talks to the website over Wi-Fi
RC522 RFID module	13.56 MHz, SPI, runs on 3.3 V (never 5 V)	Reads the tag id from a card held near it
RFID tags	13.56 MHz MIFARE 1K cards or stickers, one per hanger	Each hanger gets one. The tag id (for example F3589104) identifies the garment
Active buzzer	On GPIO 25 (+) and GND (-)	Sounds the beep patterns
LED	The on-board blue LED on GPIO 2	Blinks together with the buzzer
Jumper wires (7) and USB cable	-	Wiring and power / programming

3.5 Development and test tools

Tool	Used for
VS Code	Writing the Python, HTML, CSS and JavaScript code
Arduino IDE (esp32 board package, MFRC522 library)	Writing and uploading the ESP32 sketch; Serial Monitor at 115200 baud for settings and debugging
Google Chrome	Running the website; testing push notifications
Selenium + headless Chrome	Automatic browser checks (toasts, wardrobe, matcher, live Hangers page)
In-memory fake database (tests/fakedb.py)	Lets the 126 automated checks run without touching Supabase
Supabase dashboard (SQL Editor)	Running the four SQL files; reading data
pip-audit	Checks the pinned libraries for known vulnerabilities
ReportLab, Pillow	Making the project PDFs and the sample clothing images

4. IoT hanger-tag system

4.1 What it does

Each hanger carries an RFID tag. When the user takes a garment off the rail, the tag touches (or is held near) the reader. The reader sends the tag id to the website. The website identifies which piece it is, counts it as worn, compares it with the last seven days, and tells the user, all without typing anything.

4.2 Wiring

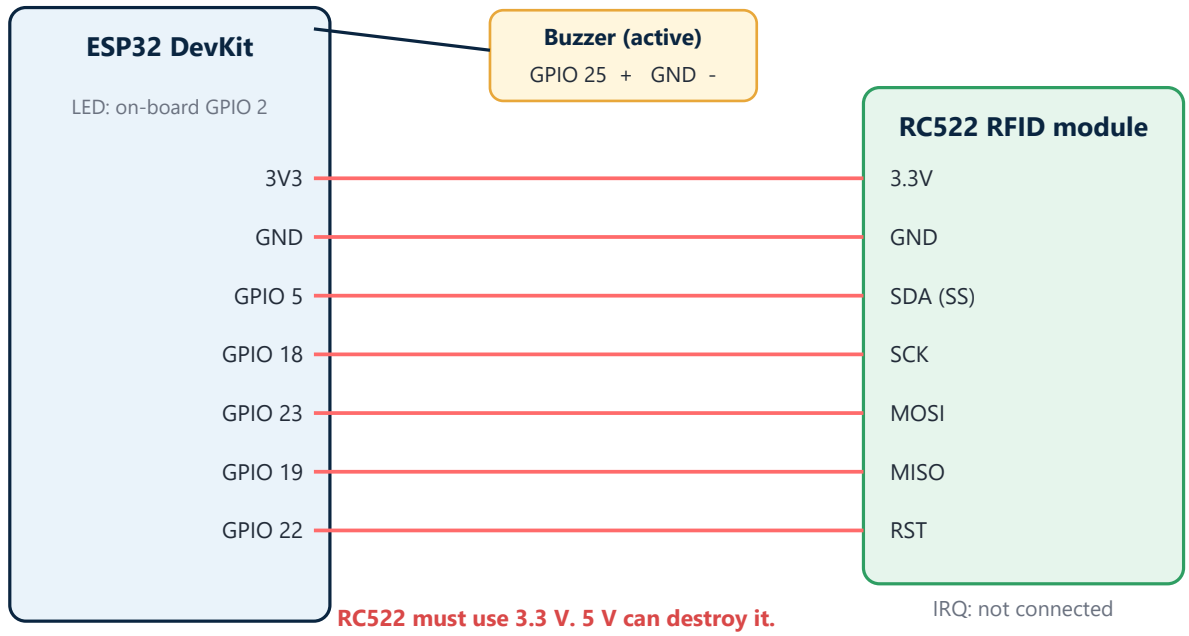


Figure 2. Wiring of the ESP32 to the RC522 module, the buzzer and the LED.

RC522 pin	ESP32 pin	Purpose
3.3V	3V3	Power (never 5 V)
GND	GND	Ground
SDA (SS)	GPIO 5	SPI chip select
SCK	GPIO 18	SPI clock
MOSI	GPIO 23	SPI data to the module
MISO	GPIO 19	SPI data from the module
RST	GPIO 22	Reset
IRQ	not connected	unused
Buzzer +	GPIO 25	Beeps (the other leg to GND)

4.3 The firmware (rfid_scanner.ino)

The sketch is about 200 lines of Arduino C++ and uses four libraries: WiFi, HTTPClient, SPI/MFRC522 and Preferences (the ESP32's permanent memory).

What the sketch does, in order

1. **Start-up:** initialises the RC522, loads the saved settings, prints the RC522 version (0x91 or 0x92 means the wiring is good), connects to Wi-Fi (2 blinks when connected).
2. **Loop:** every cycle it checks the Serial Monitor for typed commands, reconnects Wi-Fi if it dropped, and asks the RC522 whether a card is present.
3. **Card found:** converts the tag id to hex text (for example F3589104). If it is the same tag as a moment ago (within 4 seconds) it is ignored, so one swipe is sent once.
4. **Send:** HTTP POST to the server with the header `X-Device-Key` and the body `{"tag_uid":"F3589104"}`.
5. **Answer:** the server replies with a tiny JSON like `{"repeat":false,"status":"logged"}`. The sketch turns it into a sound.

Settings are changed without re-uploading the code

Open the Arduino Serial Monitor (115200 baud, line ending "Newline") and type a command. The values are saved in the ESP32's permanent memory and survive power-off. The defaults written in the sketch (DEFAULT_SSID and so on) are only a fallback when nothing is saved.

Command	Meaning
ssid My Hotspot	Set the Wi-Fi name (must be a 2.4 GHz network)
pass mypassword	Set the Wi-Fi password
url http://192.168.1.106:5000/api/iot/scan	The laptop's address on that network (find it with ipconfig); not "localhost"
key PASTE_DEVICE_KEY	The secret key of this reader (from the Hangers page)
show	Print the saved settings (secrets are hidden after 3 characters)
restart	Reboot so new settings are used

Beep patterns

Sound	Server answer	Meaning
1 short beep	status = logged	The piece was counted as worn
2 short beeps	status = unknown	New or unlinked tag: link it on the Hangers page
1 long beep	repeat = true	Worn this week already (piece or outfit): check the notification
3 quick beeps	HTTP error	Something went wrong: Wi-Fi, wrong key, too many scans

4.4 The scan, step by step

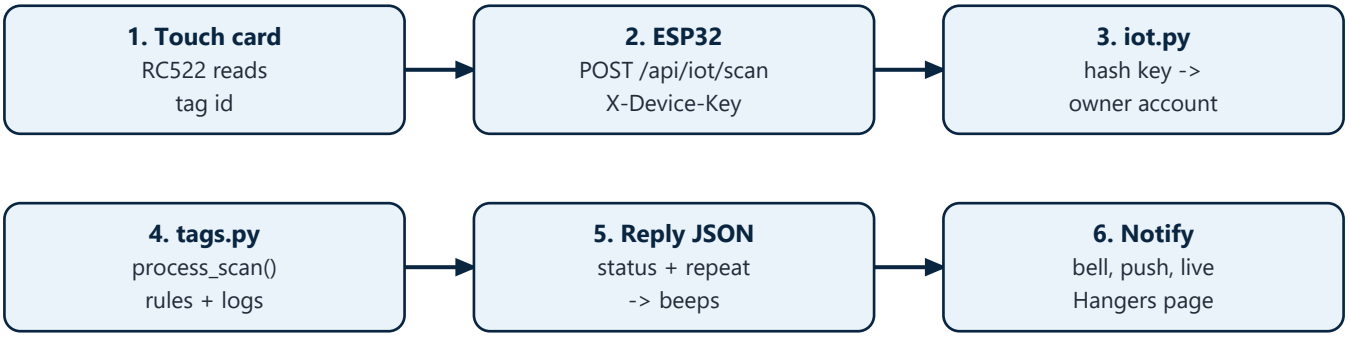


Figure 3. What happens from the card touch to the notification.

Step	Where	Detail
1	RC522 / ESP32	Reads the card id, ignores the same card for 4 seconds
2	ESP32 to server	POST /api/iot/scan with X-Device-Key. Plain HTTP on the local network during development
3	iot.py	Key length check, SHA-256 hash, look up the device. The reader never says which user it is: the key decides the owner. Rate limits: 120 per minute per IP, 30 per minute per key. A wrong, missing or removed key always gets the same 401
4	tags.process_scan()	Debounce 5 seconds per tag; find the tag in this owner's account; unknown tag: store it and notify; linked tag: count the wear (once per day), run the repeat rules, group into a hanger look, notify
5	Reply	Tiny JSON with status and repeat. The ESP32 beeps accordingly
6	notify.py	Stores a notification (bell), sends a Web Push (pop-up), and the open Hangers page notices the change within about 3 seconds and refreshes itself

4.5 The rules inside process_scan

Rule	How it works
Debounce	A reader often reports one swipe two or three times. The same tag within 5 seconds is ignored
Unknown tag	A tag that is not linked yet is stored once (the Hangers page never lists it twice) but every real read creates a notification, so the user is told each time until they link it
Wear counted once per day	Wear count goes up at most once per calendar day for a piece. A second tap on the same day says "already logged / already in today's outfit" instead of counting again
One-week rule (piece)	If the piece was last worn between 1 and 7 days ago the user is warned and up to two fresh alternatives of the same category are suggested. Nothing is exempt (shoes and accessories follow the same rule)
One-week rule (whole outfit)	If the pieces taken out today include every piece of an outfit worn earlier this week, a warning says the outfit repeats. It speaks only at the scan that completes the repeat
Hanger look	Pieces scanned within 2 hours of each other become one "Hanger look" (2 to 6 pieces) in History
Today and yesterday	Counted on the laptop's local clock, so "yesterday" starts at the user's midnight
Limits	5 readers and 300 tags per account

4.6 Pairing a reader and linking tags

1. Log in and open the **Hangers** page. Type a name and click **Pair a reader**.
2. The server creates a random 256-bit key and shows it **once**. Only its SHA-256 hash is stored. Copy the key into the ESP32 with the Serial command `key ....`
3. Hold a new card to the reader once. It appears in step 2 of the Hangers page as a waiting tag (and you hear 2 short beeps).
4. Choose the piece of clothing in the drop-down and click **Link**. One tag per piece: linking again replaces the old tag.
5. From now on, scanning that card logs that piece as worn.

4.7 Tags belong to the account, not to the card

The database key of the tags table is the pair (user_id, tag_uid). Two accounts can therefore use the same physical card with different meanings: a card can be "Nike shoes" in one account and "G-Shock watch" in another. A reader belongs to exactly one account (through its key), so scans from that reader always go to that account. To use the same ESP32 for another account, pair a new reader under that account and change the key on the ESP32 with the Serial command `key`.

4.8 Notifications

Channel	How it works	File
Bell	Header icon with unread count. The page asks /api/notifications every 15 seconds and also when the tab becomes visible again. Opening the bell marks items as read	notify.py, notifications.js
Browser push	The user clicks "Turn on browser notifications" in the bell. The browser creates a subscription; the server stores it and later sends signed, encrypted pushes. A service worker shows the pop-up and tells open tabs to refresh the bell at once	notify.py, sw.js
In-page pop-up (toast)	Small message that stays 7 seconds (9 seconds for a problem), pauses while the mouse is on it, and has a close button	toast.js
Live Hangers page	Every 3 seconds the page asks /api/tags/feed for a fingerprint of the tag list; if it changed, the page shows the scan message and reloads itself. It never reloads while the user is typing or while the one-time key is displayed	hangers.js, tags.py
Reminder	At most once a week: "You have not worn X in 60 days"	notify.py

Pop-up titles tell the kind of event: **Hanger scan**, **New hanger tag**, **Worn this week**, **Outfit repeat**, **Already worn today**.

4.9 Working without hardware

- **Simulate scan** on the Hangers page runs exactly the same code as a real reader (process_scan), for demos before the hardware is ready.
- `tools/fake_reader.py` sends one scan from the command line, like an ESP32 would.
- `tools/pretend_worn.py` pretends a piece was last worn N days ago, to demonstrate the repeat warning.
- `tools/make_vapid_keys.py` creates the key pair needed for push notifications.

4.10 Screenshots of the finished IoT feature

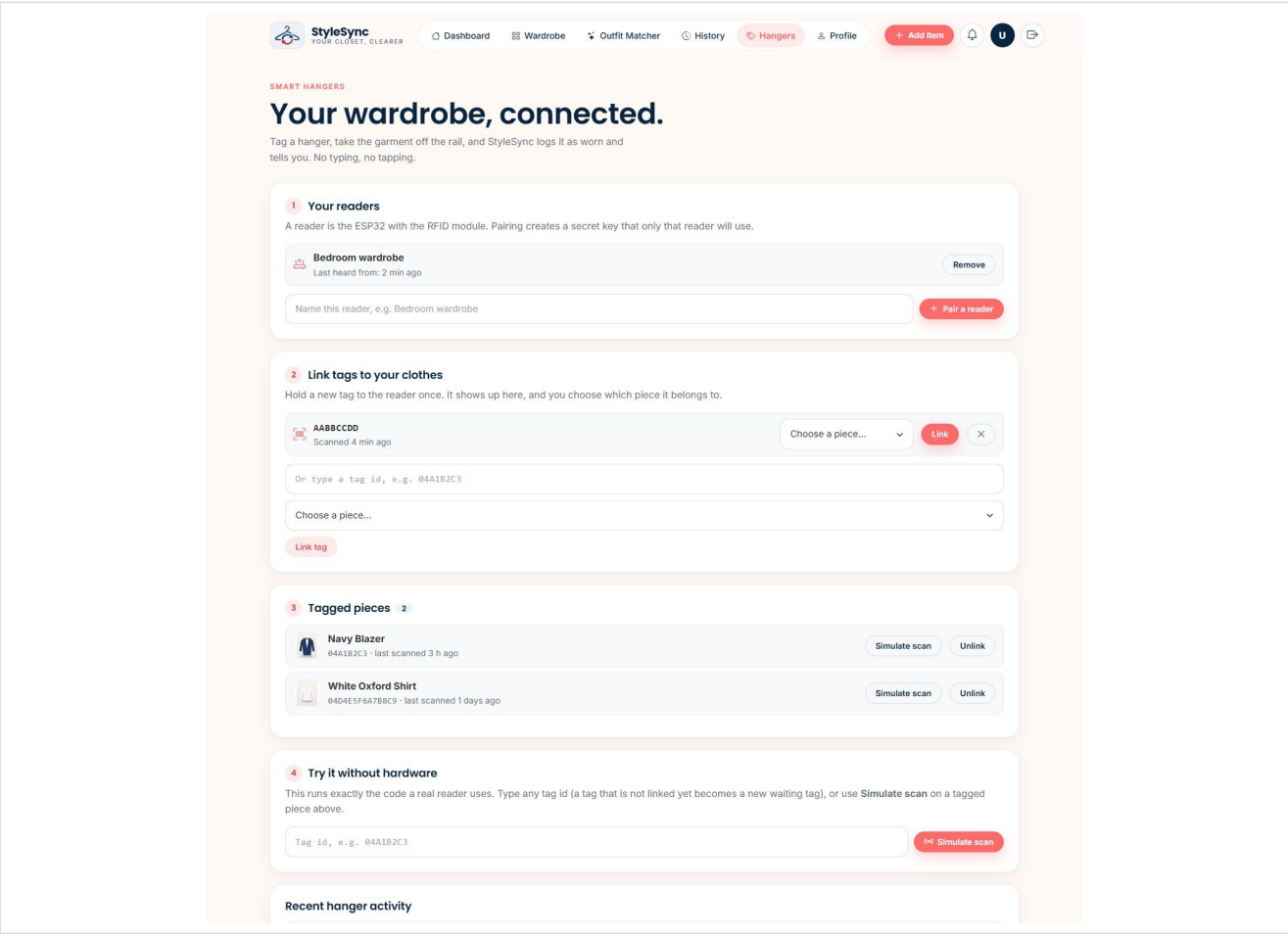


Figure 4. The Hangers page: readers, link tags, tagged pieces, try without hardware, recent activity.

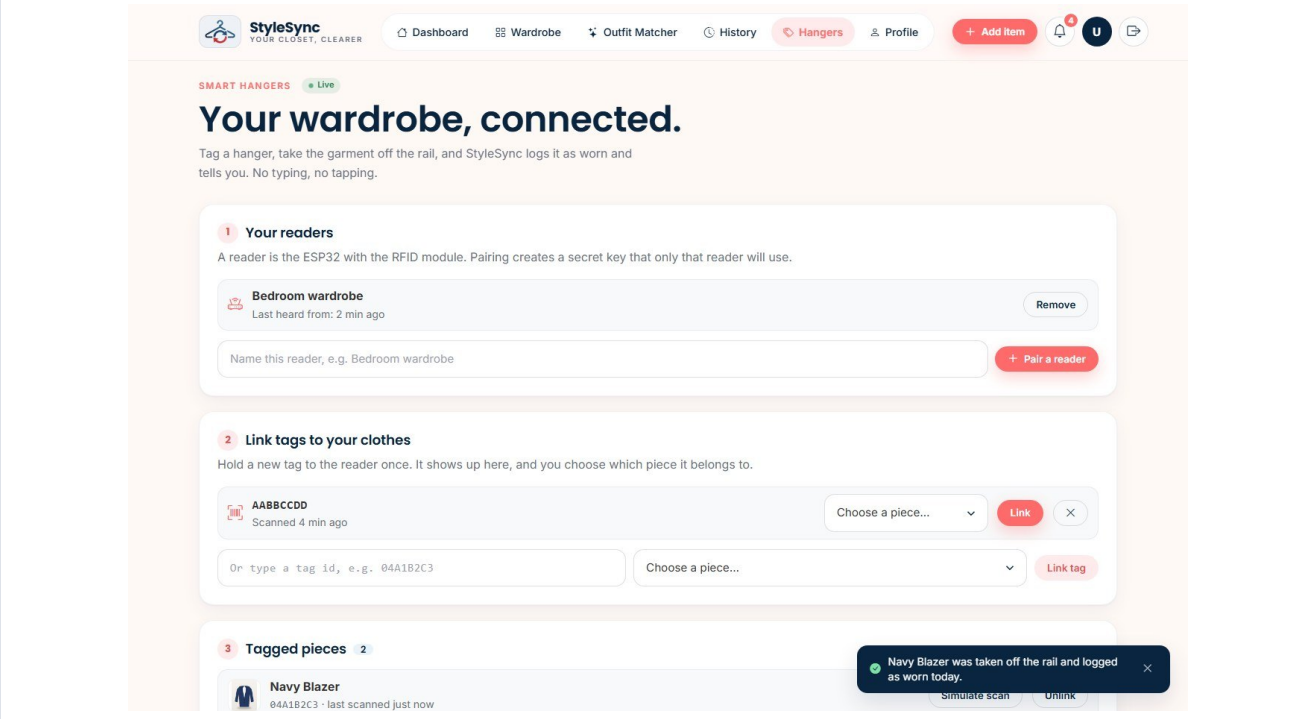


Figure 5. The Hangers page with the green Live badge (updates itself after a scan).

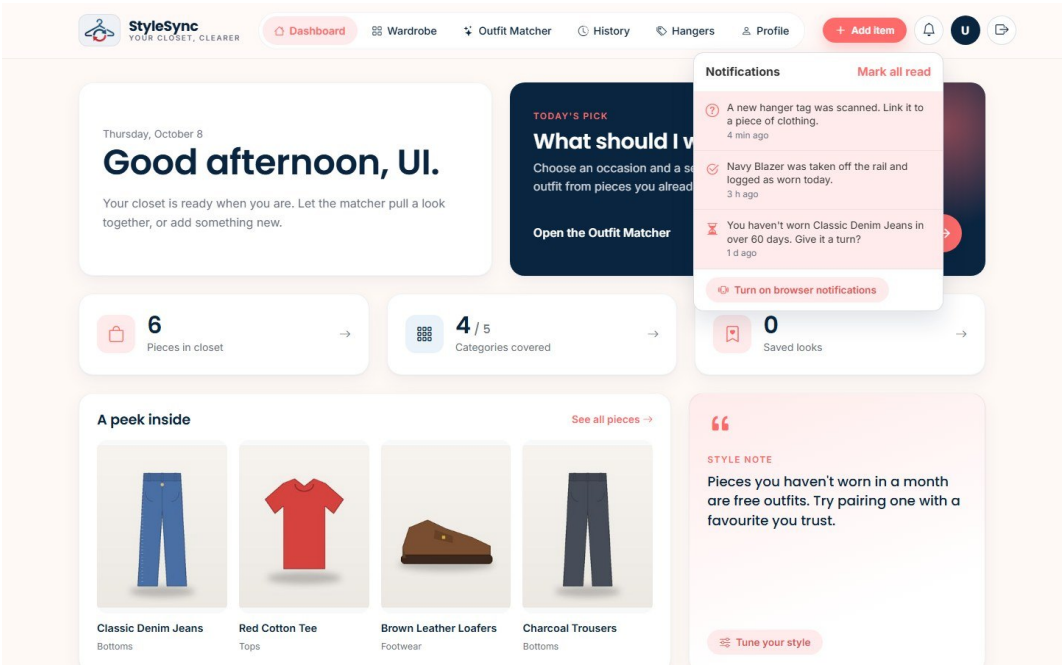


Figure 6. The notification bell with scan messages.

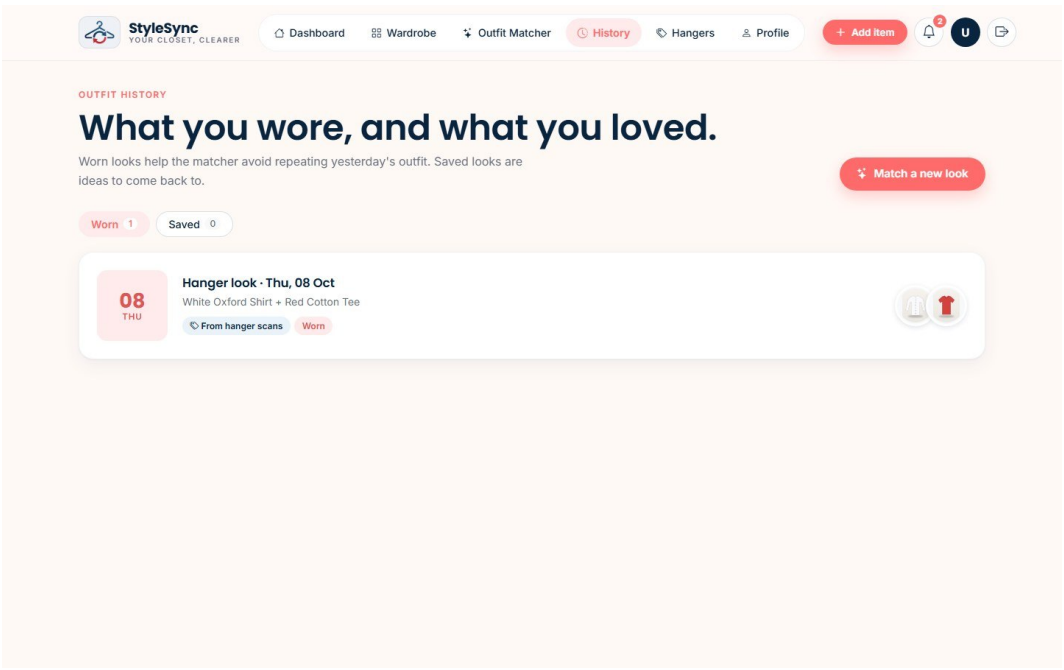


Figure 7. History showing a Hanger look created by scanning several pieces.

> **BE HONEST ABOUT THIS**
>
> Photographs of the real ESP32 + RC522 + buzzer and a Serial Monitor screenshot were not available when this document was made. The report author should take them (the wiring in Figure 2 shows what to photograph) together with Chrome's real push pop-up.

5. Back-end implementation

5.1 Module by module

File	Purpose
app.py	Creates the Flask app, registers the six blueprints, sets cookie and size limits, adds security headers and the per-request CSP nonce, defines the page routes, error handlers (generic messages), the "ago" template filter and the service-worker route
config.py	Startup safety checks: refuses to start if a secret is missing, a placeholder, too short, swapped, or if debug is on in production
db.py	One connection to Supabase using the service-role key (server only)
auth.py	signup, login, logout; @login_required; session handling; the same error for every login failure
validators.py	Server-side checks: email, name, password (8+ characters, a letter and a number, at most 72 bytes), item fields, hidden control characters stripped
extensions.py	The CSRF protector and the rate limiter, shared by all files
items.py	List, add (with photo) and delete wardrobe pieces; 300 pieces per user
images.py	Photo handling with Pillow: open (a renamed non-image fails), convert to a fresh JPEG (removes hidden data and GPS), shrink, store with a random name in the garment-images bucket
options.py	The fixed lists (categories, occasions, seasons, styles, colours with a colour family) and the one-week constants
outfit_engine.py	The outfit matcher (chapter 6)
outfits.py	API for suggest, save look, wear this today, remove saved look; history; keeps wear counts; adds tag status to suggestions
tags.py	The heart of IoT: process_scan, link, forget, simulate, feed, page data (chapter 4)
iot.py	The scan route for readers and the pairing routes (add and remove reader)
notify.py	Bell API, push subscription API, create() that stores and pushes, the 60-day reminder, the SSRF allow-list for push hosts

5.2 Pages and API routes

Method	Route	What it does
GET	/	Landing page with login and sign-up
GET	/dashboard	Home after login
GET	/wardrobe	Gallery
GET	/wardrobe/add	Add-piece page / modal
GET	/matcher	Outfit Matcher
GET	/history	Worn, saved and hanger looks
GET	/hangers	Readers, tags, activity
GET	/profile	Favourite styles and colours
GET	/sw.js	Service worker (served from the root so it can control the whole site)
POST	/auth/signup, /auth/login, /auth/logout	Account actions
POST / DELETE	/api/items, /api/items/<id>	Add piece (multipart with photo) / delete piece
POST	/api/outfits/suggest	Build an outfit for the chosen occasion and season
POST / DELETE	/api/outfits, /api/outfits/<id>	Save or wear a look / remove a saved look
POST / DELETE	/api/devices, /api/devices/<id>	Pair a reader (key shown once) / remove a reader
POST	/api/iot/scan	Called by the ESP32 (device key, no cookie, CSRF-exempt)
POST	/api/tags/link, /api/tags/simulate	Link a tag to a piece / simulate a scan
DELETE	/api/tags/<tag>	Unlink or forget a tag
GET	/api/tags/feed	Fingerprint for the live Hangers page (polled every 3 seconds)
GET / POST	/api/notifications, /api/notifications/read	Bell list / mark as read
GET / POST	/api/push/key, /api/push/subscribe, /api/push/unsubscribe	Browser push registration

Every route except the landing page, the login/sign-up routes and the device scan route requires a logged-in user. Every change (POST or DELETE) needs a CSRF token, except the device scan route which uses the device key instead.

5.3 Authentication and sessions

- Passwords are sent to Supabase Auth, which stores a bcrypt hash. The application never stores, logs or returns a password.
- A successful login clears the old session first (stops session fixation), then stores the user id and name in a signed cookie.
- The cookie is HttpOnly, SameSite=Lax, and in production Secure with the __Host- prefix. It expires after 2 hours.

- Every login failure shows the same message, so an attacker cannot learn which emails exist.

5.4 Photo handling

1. The user picks a JPG, PNG or WebP (up to 6 MB per request).
2. Pillow opens it: a renamed text file fails here.
3. It is converted to a new JPEG, which removes hidden metadata and GPS data, and shrunk.
4. It is stored in Supabase Storage under a random name; only the URL is saved with the piece.
5. Deleting a piece also deletes its photo.

6. The outfit recommendation algorithm

The Outfit Matcher is rule-based, not machine learning, on purpose: every score is a number the developer chose and every decision can be explained to the user and to the examiner. The code is in `outfit_engine.py` (the top of the file has a short explanation).

6.1 Pipeline



Figure 8. The six stages of the Outfit Matcher.

6.2 Inputs

Input	Choices
Occasion (required)	Casual, Work, Formal, Party, Ethnic
Season (required)	Summer, Winter, Monsoon, All-season
Style (optional)	Casual, Formal, Minimalist, Streetwear, Classic, Bohemian, Athleisure, Ethnic
Colour mood (optional)	Neutrals, Monochrome, Earthy, Bold
Profile favourites	The user's favourite styles and colours (a small bonus)

6.3 Points for one piece

Rule	Points
Occasion equals the chosen occasion	+3
Occasion is a related one (Work to Formal, Casual to Party and so on)	+1
Style equals the chosen style	+2
Colour family equals the chosen colour mood	+2
Style or colour is one of the user's favourites	+1 each
Made specifically for the chosen season (not just all-season)	+1
Rarely worn (fewer than 5 wears)	+0.5
Worn in the last 7 days	-3

6.4 Occasion comes first

For each slot (top, bottom, shoes) only the pieces *closest* to the chosen occasion are kept: exact matches first; if none exist, the nearest related occasion; and so on. Freshness and the one-week rule choose *within* that group and can never beat the occasion. This fixed a real bug where a Party request showed Casual clothes because rarely-worn Casual pieces earned more points.

6.5 Scoring a whole outfit

Part	Effect
Slot weights	Top, bottom and shoes count fully; a jacket (layer) counts 0.7 and an accessory 0.5
Colour palette	All neutral: +2. One statement colour on neutrals: +3. Several colours of the same family: +1. Two clashing colours: -2. Three or more: -4
Style unity	All pieces share one style: +2. Two styles: +1
Dress code	For Formal and Work, each sporty piece (Streetwear, Athleisure) costs -2
Repeat of last week	If the clothes include every piece of a look worn in the last 7 days: -10 (only chosen when nothing else is possible)
Match percent	Score divided by the best score possible for those choices, kept between 45% and 99%

6.6 Choosing the look

- Only the best 6 pieces per slot are combined, which keeps the number of combinations small and fast.
- Looks already shown for these choices are skipped until all have been shown (then it starts over). Shuffle therefore always shows something new.
- Among the looks within 2.5 points of the best unseen look, one is picked at random, so results vary but stay good.
- If a core slot is empty (no top, bottom or shoes for that season), the answer lists what is missing instead of an outfit.

6.7 The explanation

Up to five plain sentences are generated, for example: "4 of 4 pieces are tagged for work.", "A calm neutral palette that goes with everything.", "Navy Blazer adds a layer as an extra layer.", "Red Cotton Tee is due a turn: you've worn it only 3 times." If a related occasion had to be used, or the look repeats this week, that is stated first.

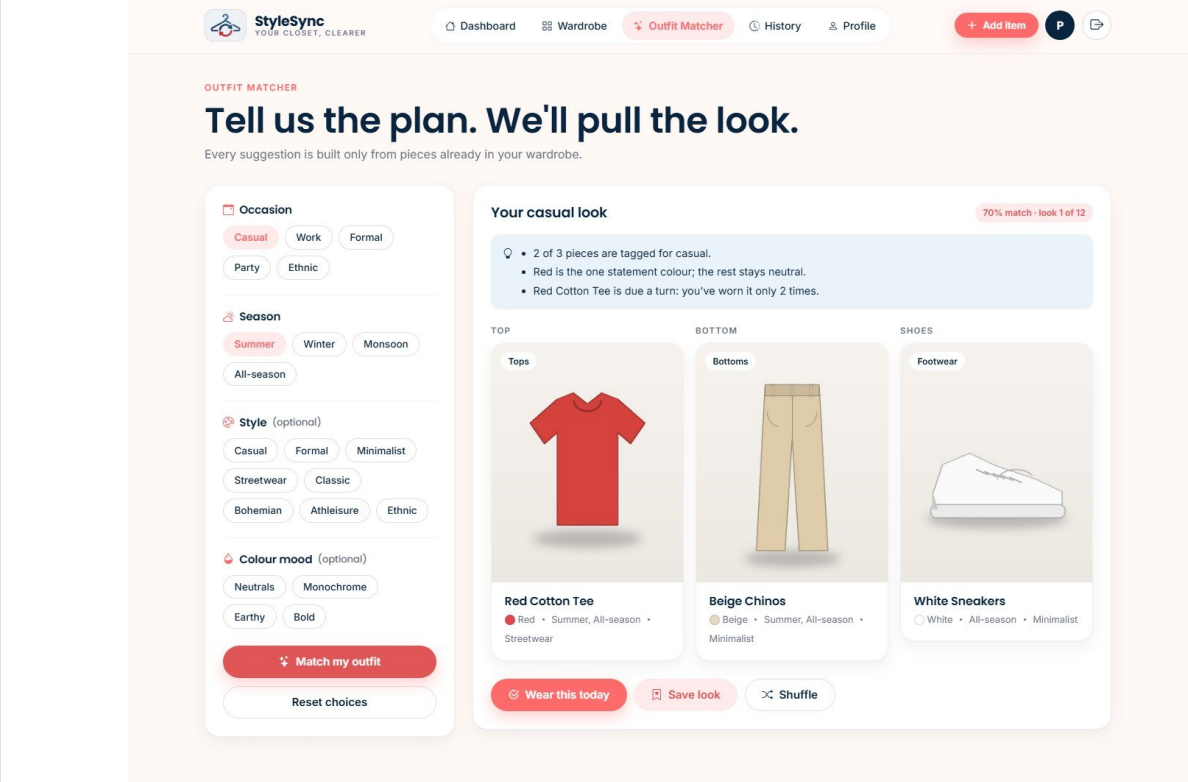


Figure 9. A suggested look with match percentage, reasons and the Wear this today / Save look / Shuffle buttons.

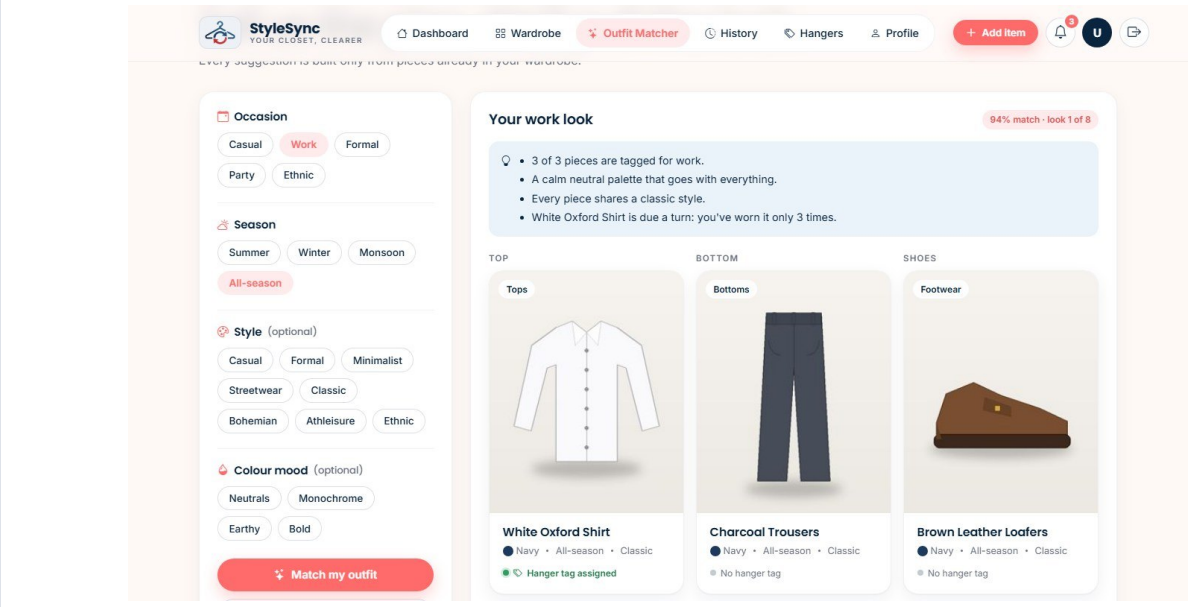


Figure 10. Suggested pieces show whether a hanger tag is assigned (green) or not (grey).

7. Database design

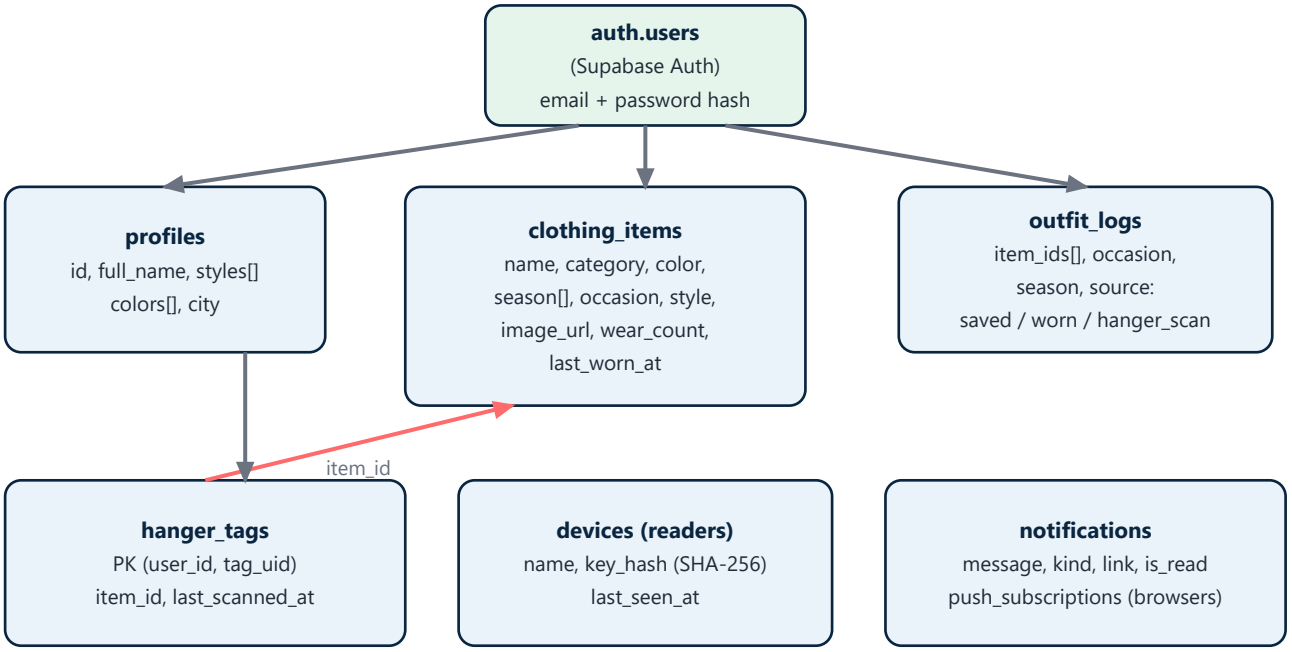


Figure 11. The tables and how they relate (all except auth.users carry user_id).

Table	Main columns	Notes
profiles	id (= auth user), full_name, styles[], colors[], city	Created automatically by a trigger when a user signs up
clothing_items	id, user_id, name, category, color, season[], occasion, style, notes, image_url, wear_count, last_worn_at	category is one of top, bottom, outerwear, footwear, accessory; name at most 60 characters
outfit_logs	id, user_id, item_ids[], occasion, season, source, created_at	source is saved, worn or hanger_scan
hanger_tags	(user_id, tag_uid) primary key, item_id, last_scanned_at	tag_uid must match 8 to 20 hex characters. Deleting a piece sets item_id to null
devices	id, user_id, name, key_hash (unique), last_seen_at	Only the SHA-256 hash of the key is stored
push_subscriptions	id, user_id, endpoint (unique), p256dh, auth	One row per browser
notifications	id, user_id, message, kind, link, is_read, created_at	kind is info, scan, unknown_tag or nudge; a link must start with / (never another website)

7.1 Security of the data

- Row Level Security is switched on for every table. No policies exist and the public roles (anon, authenticated) have no access at all: the public Supabase key cannot read or write anything.
- Only the Flask server, using the service-role key, reads and writes, and every query filters by the logged-in user's id.

- Deleting an account removes its rows (on delete cascade).

7.2 SQL files (run once, in this order, in the Supabase SQL Editor)

File	What it does
database/schema.sql	Creates the first tables, row-level security, the sign-up trigger and the server permissions
database/hardening.sql	Locks the public roles out, adds extra checks and limits
database/iot.sql	Adds readers, push subscriptions, notification kinds and links, tag format checks
database/tags_per_user.sql	Changes the tag key to (user_id, tag_uid) so the same card works in several accounts

8. Front-end design

8.1 Design system

The theme is calm and fashion-oriented: a navy brand colour, a coral accent, soft shadows and rounded cards. All values live as CSS variables at the top of style.css, so changing the theme means editing one block.

Token	Value
Primary (navy)	#0A2540
Accent (coral)	#FF6B6B
Secondary (sky)	#A0C8E8
Background	#FFF9F6
Success / danger / warning	#2F9E63 / #E5484D / #E8A73A
Fonts	Poppins (headings), Inter (body)
Radius	8, 14, 20 px and pill
Navigation height	72 px

8.2 Pages

Page	Template	Script	What the user does
Landing (login / sign-up)	landing.html	landing.js	Create an account or log in; the password rule (8+ characters, letter and number) is checked in the browser and again on the server
Dashboard	dashboard.html	dashboard.js	Greeting, statistic cards that link to pages, shortcut to the matcher, style tip
Wardrobe	wardrobe-gallery.html + macros.html	wardrobe-gallery.js, add-item-modal.js	Filter by category, search, add and delete pieces; see tag status and wear count on every card
Outfit Matcher	outfit-matcher.html	outfit-matcher.js, sparkles.js	Pick occasion and season, get a look, shuffle, save, wear today
History	history.html	history.js	Worn, saved and hanger looks; remove saved
Hangers	hangers.html	hangers.js	Pair readers, link tags, simulate a scan, see activity, live updates
Profile	profile.html	profile.js	Favourite styles and colours
Error pages	error.html	-	Friendly generic messages for 404, 429, 500 and so on

8.3 Shared scripts

File	Role
app.js	api() helper that adds the CSRF token and parses answers; escapeHtml(); renderItemCard() used by the matcher; setupSinglePillGroup(); toast-after-reload
toast.js	showToast(): in-page messages with close button and hover pause (text inserted as text, never HTML)
notifications.js	The bell: polling, unread badge, list, push subscription button
sw.js	Service worker: receives push, shows the pop-up, opens the right page, tells tabs to refresh the bell
sparkles.js	The sparkle animation when a match is found (respects reduced-motion settings)

8.4 Principles that were followed

- Plain JavaScript and server-rendered pages: nothing to install, easy to explain.
- Every value coming from users is escaped on output (Jinja auto-escape on the server, escapeHtml() in JavaScript).
- Content-Security-Policy with a per-request nonce, and pinned CDN files with integrity hashes (details in chapter 9).
- Responsive layout (mobile-first) and visible focus states; icons have aria-hidden and buttons have labels.
- Fixed dropdown lists come from options.py and are shared with JavaScript, so lists are written once.

8.5 Screenshots

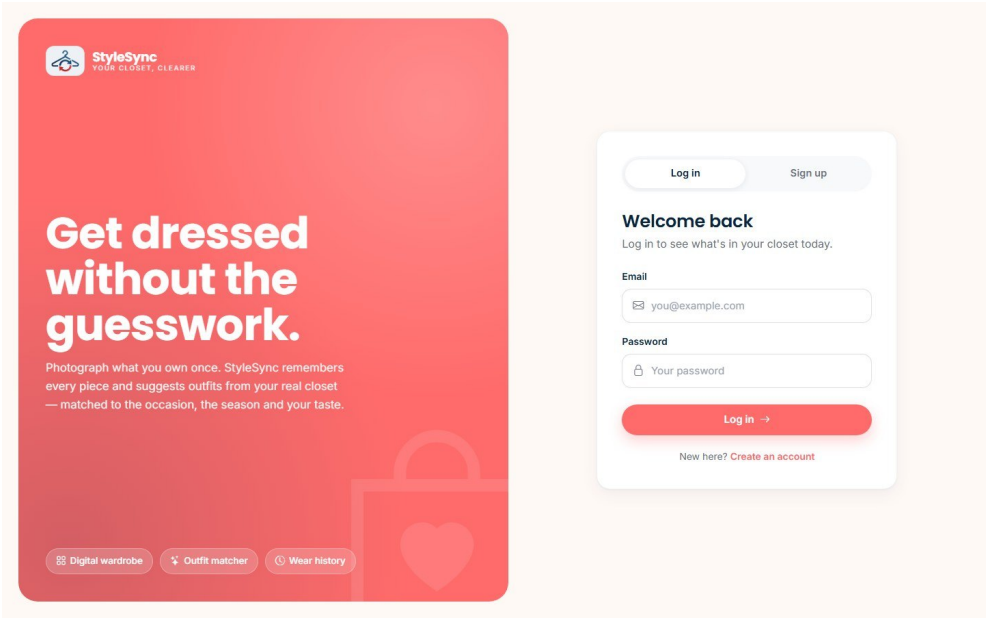


Figure 12. Landing page with login.

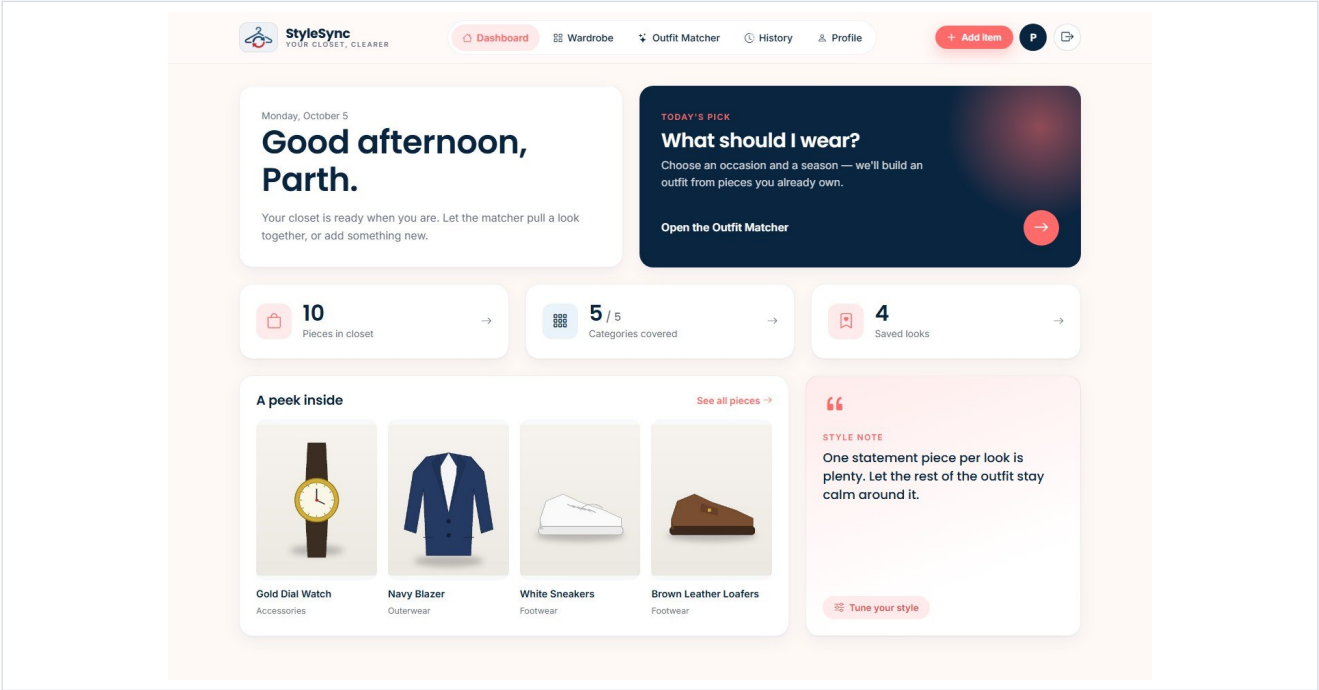


Figure 13. Dashboard.

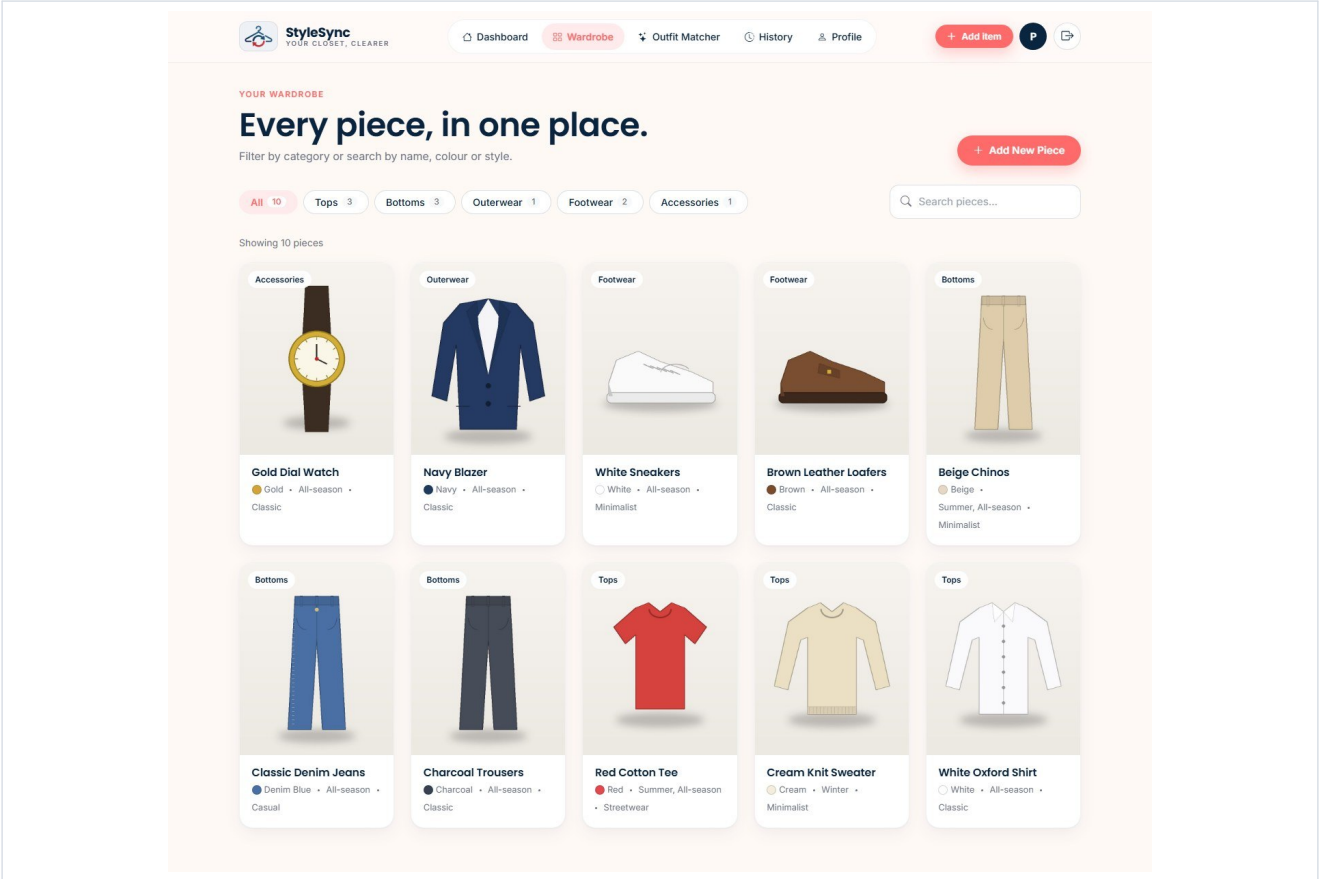


Figure 14. Wardrobe gallery.

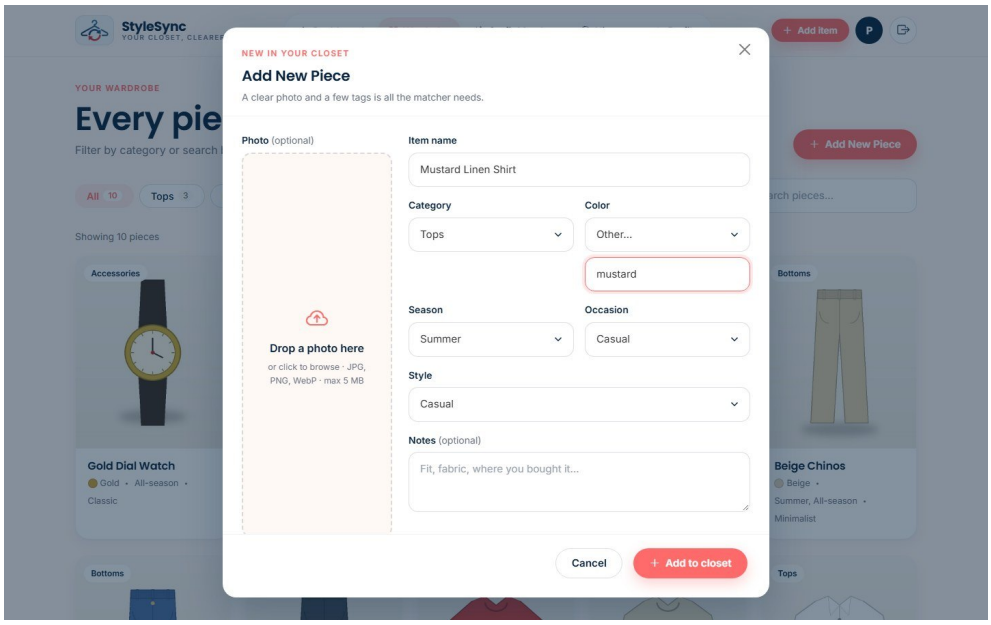


Figure 15. Add-piece popup.

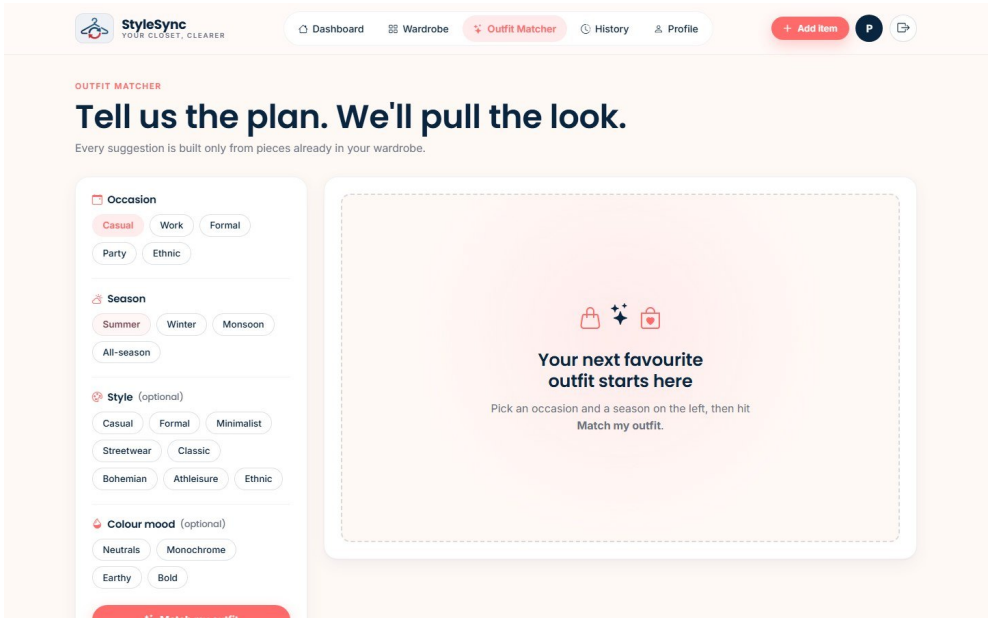


Figure 16. Outfit Matcher: choices.

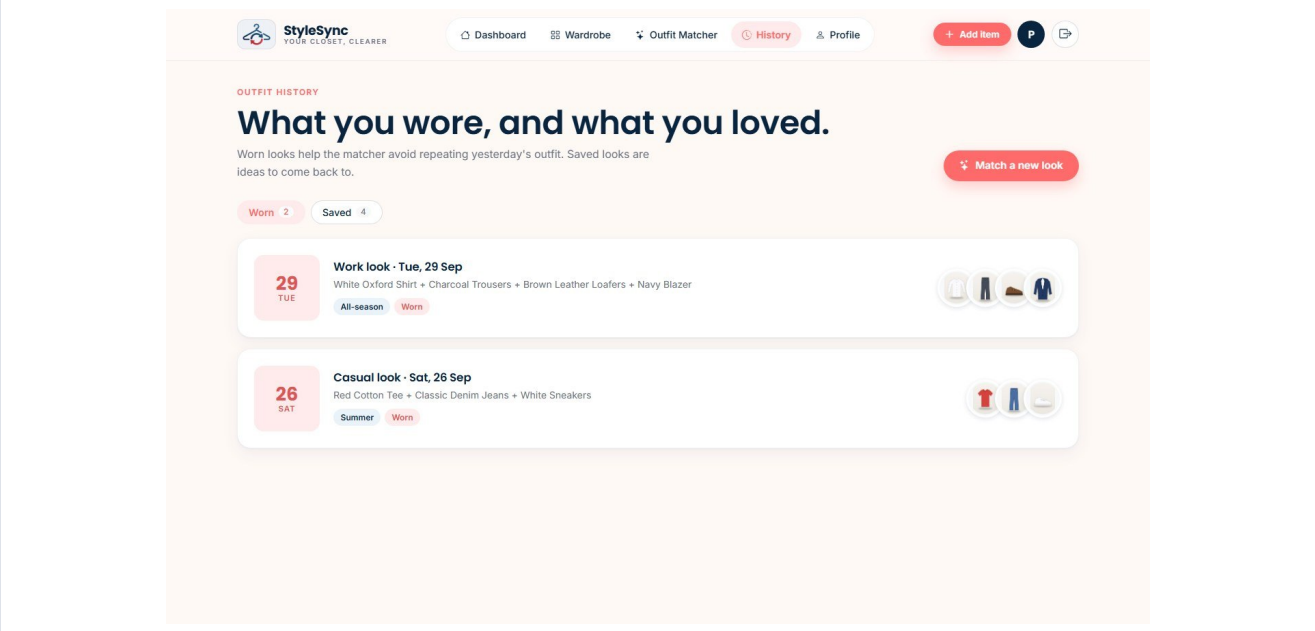


Figure 17. History: worn looks.

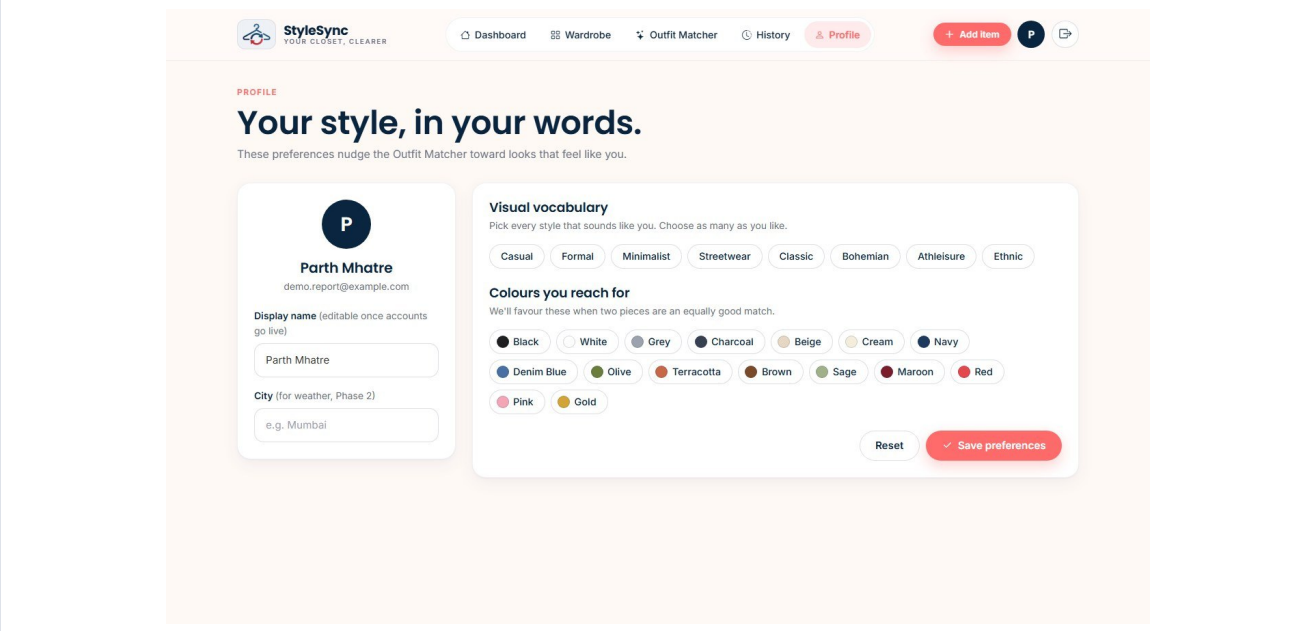


Figure 18. Profile.

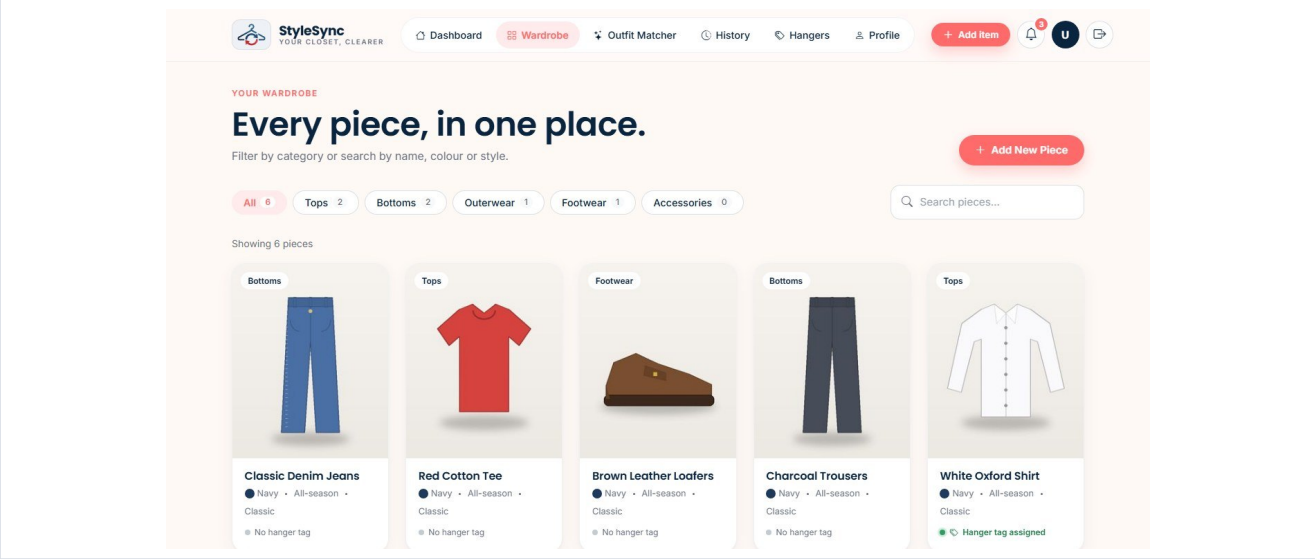


Figure 19. Wardrobe cards with the green "Hanger tag assigned" line, grey "No hanger tag", and the wear line.

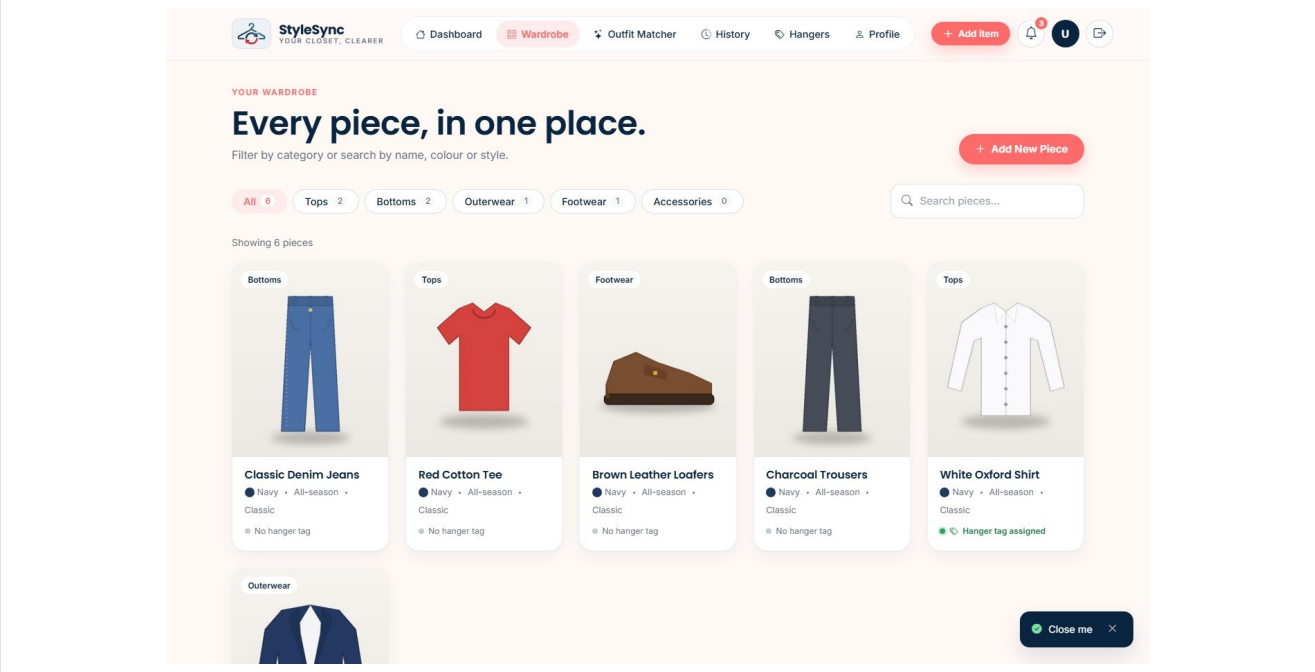


Figure 20. A pop-up message with its close button.

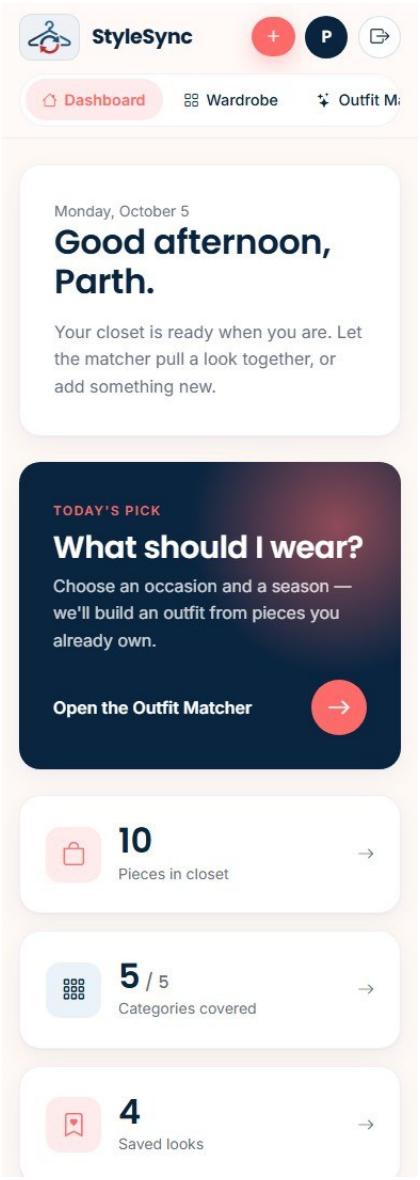


Figure 21. Phone layout of the dashboard.

9. Security

Security was a stated requirement from the beginning: server-side validation, rate limiting, password hashing, generic error messages and proper authentication. The table shows what is protected and where. (The full detail is in the file SECURITY.md.)

Threat	Defence	Where
Guessing passwords	Login limited to 5 per minute and 30 per hour per IP, plus 10 per hour per email; same error for every failure	auth.py, extensions.py
Stored passwords stolen	Passwords are only ever stored as bcrypt hashes by Supabase Auth	Supabase Auth
Stolen or forged cookie	Signed cookie, HttpOnly, SameSite=Lax, Secure + __Host- in production, 2-hour life, session cleared at login and logout	app.py, auth.py
One user reading another's data	Every query filters by user id; foreign ids return "not found"; RLS on with no public access	all modules, database/
Bad input	All rules repeated on the server; fixed lists; hidden characters stripped; length caps	validators.py
Script injection (XSS)	Jinja auto-escaping, escapeHtml() in JavaScript, textContent in toasts, strict Content-Security-Policy with nonce	templates, JS, app.py
Forged requests (CSRF)	CSRF token on every POST/DELETE (the device route uses its key instead)	extensions.py
Malicious uploads	Pillow re-encodes to a clean JPEG, random file names, size limit 6 MB	images.py
Flooding	Per-route rate limits, per-user caps (300 pieces, 2000 history rows, 5 readers, 300 tags), 5-second tag debounce	extensions.py, tags.py
Leaked keys	Secrets only in .env (git-ignored); startup refuses placeholders or swapped keys; service key never reaches the browser	config.py
Fake readers	256-bit device key shown once, only its SHA-256 hash stored, revocable, can only report scans for its owner, 30 scans/minute per key	iot.py
Server-side request forgery via push	Push endpoints must be https and on a short list of real push hosts	notify.py
Notification links opening other sites	Links must start with / and the service worker opens only our own address	iot.sql, sw.js
Clickjacking and sniffing	X-Frame-Options DENY, nosniff, Referrer-Policy, Permissions-Policy, Cross-Origin-Opener-Policy; HSTS in production	app.py
Vulnerable libraries	Exact versions pinned; pip-audit reports none	requirements.txt

9.1 Known limits (honest list for the report)

- Rate-limit counters are kept in memory and reset when the server restarts (use Redis for production).

- The scan request travels over plain HTTP on the local network; a real deployment needs HTTPS. A captured scan could be replayed on a hostile network (signed timestamps would stop that).
- RFID tag ids are not secret and can be copied by anyone with a reader; a scan is a logging convenience, not proof of identity.
- Sign-up has no email verification (postponed to the major project).
- Photos are public by URL (random, unguessable names).
- A stolen cookie works until it expires (2 hours).

10. Testing and results

10.1 Automated checks: 126 passed, 0 failed

The script `tests/run_iot_checks.py` runs 126 checks against an in-memory fake database, so it needs no Supabase tables and changes no real data. Run: `venv\Scripts\python.exe tests\run_iot_checks.py`.

Group	What is checked
Pairing and scans	Key shown once and strong; wrong, missing, short or revoked key gives 401; scan logs a piece; limits on readers and tags
Isolation	A user cannot read, link or scan another user's tags or pieces
Notifications and push	Bell list, mark read, reminder, push subscription accepts only real push hosts, a real encrypted push is decrypted by a fake push service
Pages and static files	Pages render and require login; service worker is served
One-week rule	Piece repeat, outfit repeat, nothing exempt, same-day taps, alternatives suggested
Matcher	Occasion first (Party request returns Party pieces), repeat penalty, seen / exhausted shuffle
Same day and midnight	Worn 10 minutes before midnight counts as yesterday; 10 minutes after midnight is today; titles of pop-ups
Per-account tags	The same card in two accounts works independently; unlinking in one leaves the other
Beep data, wear line, feed, matcher tags	The JSON the reader parses, the wear line text, the live feed fingerprint, tag flags on suggested pieces
Every new route	Login required, CSRF required, rate limit works (a noisy reader gets 429)

10.2 Browser checks (headless Chrome, 24 checks in total)

- Pop-ups stay 7 seconds, have a working close button, pause on hover, show text as text (no HTML injection).
- Wardrobe shows green and grey tag status and the wear line on every card.
- Matcher cards show the tag line; Hangers page shows the Live badge.
- After a scan the Hangers page reloads itself within 10 seconds and shows the pop-up; it does not reload while the user is typing.
- No console errors.

10.3 Real hardware tests (done by the project owner)

- ESP32 + RC522 + buzzer on a home network and a phone hotspot; two real cards read (ids F3589104 and 2461D605).
- Unknown card reported, linked to a piece, then logged as worn; browser notification received; tag assigned in one account appears as new in another account.
- Server tested live against the real Supabase project earlier in development.

BE HONEST ABOUT THIS

Not verified by the assistant who helped write the code: the final sketch with the three different beep patterns was not compiled on the development PC (no Arduino compiler installed). It must be uploaded once and listened to. Also verified only on Windows with Chrome and one ESP32 DevKit.

11. Installation, running and demonstration

11.1 First-time setup

1. Install Python 3.11 or newer and create a free Supabase project.
2. Copy `.env.example` to `.env` and fill: SUPABASE_URL, SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY, FLASK_SECRET_KEY (32+ characters) and, for push, VAPID_PUBLIC_KEY, VAPID_PRIVATE_KEY and VAPID_SUBJECT (made with tools/make_vapid_keys.py). **Never share .env.**
3. In the project folder: `python -m venv venv`, then `venv\Scripts\pip install -r requirements.txt`.
4. In Supabase, create a public Storage bucket named `garment-images`.
5. In the Supabase SQL Editor run, in this order: schema.sql, hardening.sql, iot.sql, tags_per_user.sql (from the database folder).
6. Start the site: `$env:HOST="0.0.0.0"; venv\Scripts\python.exe app.py` and open http://127.0.0.1:5000. (HOST 0.0.0.0 lets the ESP32 reach the laptop.)

11.2 Setting up the reader

1. Wire the ESP32 and RC522 as in section 4.2. Install the Arduino IDE, the esp32 board package (Espressif) and the MFRC522 library.
2. Choose board "ESP32 Dev Module" and the right COM port. Open firmware/rfid_scanner/rfid_scanner.ino and click Upload.
3. Open Serial Monitor (115200, line ending Newline). Type `ssid`, `pass`, `url http://<laptop-ip>:5000/api/iot/scan` and `key ...` (the key from the Hangers page), then `restart`.
4. Both devices must be on the same Wi-Fi, and the Wi-Fi must be 2.4 GHz. Find the laptop's address with `ipconfig`.

11.3 College demonstration with a phone hotspot

1. Turn on the phone's hotspot (2.4 GHz). Connect the laptop to it.
2. Run ipconfig and note the laptop's IPv4 address.
3. In the ESP32 Serial Monitor: ssid, pass, url (with the new address), then restart.
4. Start the website with HOST 0.0.0.0 and demonstrate.

11.4 Three-minute demonstration script

1. Sign up and add 4 or 5 pieces (use the sample images in docs/report_assets/samples).
2. Outfit Matcher: choose Party and All-season, show the match percentage and the reasons, press Shuffle.
3. Hangers: pair a reader; scan an unknown card (2 beeps, it appears by itself on the page); link it to a piece.
4. Scan again: 1 beep, bell notification, browser pop-up.
5. Show the repeat warning: use tools/pretend_worn.py to say the piece was worn yesterday, scan again: long beep and warning with alternatives.

6. Open History to show the Hanger look. Mention security: show the 126 checks passing.

12. Problems faced and how they were solved

Problem	Cause	Solution
Bell updated slowly	Page polled every 30 seconds	Poll every 15 seconds, refresh on tab focus, and the service worker tells open tabs to refresh at once
Unknown tag notified only once	A throttle hid repeated reads	Throttle removed; every real read notifies, the tag is stored once
Party request showed Casual clothes	Rarely-worn bonus beat the occasion	Occasion-first tiers (section 6.4)
ESP32 could not connect	5 GHz network name, placeholder URL and key, empty Serial Monitor until reset	Wi-Fi settings and key saved with Serial commands; use a 2.4 GHz network
Same card unusable in a second account	The tag id was unique in the whole database	Primary key changed to (user_id, tag_uid)
Scans went to the wrong account	A reader's key belongs to one account	Pair a reader per account and change the key with the Serial command
Tapping an already-worn piece looked like an error	Old message said "taken off the rail"	New messages: already in today's worn outfit / already logged today
Wrong "yesterday"	UTC time versus local midnight	Local clock used for today and yesterday
Tests that always pass	Weak checks written with always-true conditions	Removed and replaced with real checks; fake database fixed to create a fresh query on every call
Pop-ups vanished too fast	2.8 seconds and no close button	7 s / 9 s, hover pause and a close button

13. Limitations and future work

13.1 Limitations

- One reader belongs to one account; switching accounts means changing the key.
- The site runs on the laptop with Flask's development server and is not deployed. A production setup needs a WSGI server, HTTPS, Redis for rate limits and FLASK_ENV=production.
- The ESP32 and the laptop must share a 2.4 GHz network; the laptop's address changes per network.
- The matcher is rule-based (by design) and does not use weather or body type.
- RFID ids can be copied; the system is a convenience, not an identity proof.
- Tested on Windows, Chrome and one ESP32 board.

13.2 Future work

- Laundry / "in the wardrobe" status: a second scan means the piece is back, and the matcher skips pieces that are out.
- Reader heartbeat: show readers as online or offline.
- Cost per wear: add a price to each piece and show price divided by wears.
- Several labelled readers (bedroom, laundry basket).
- A statistics page: most worn, never worn, wear by weekday.
- Weather-based morning suggestion using a free weather API and a push notification.
- Email verification at sign-up and deployment with HTTPS.

14. Appendix A: file inventory

Path	Purpose
README.md	Setup and run guide
.env / .env.example	Secrets / template
requirements.txt	Pinned Python packages
app.py, auth.py, config.py, db.py, extensions.py, validators.py	Core, login, startup checks, database link, security tools, input rules
items.py, images.py	Wardrobe and photos
outfit_engine.py, outfits.py, options.py	Matcher, history API, fixed lists
tags.py, iot.py, notify.py	Hanger tags and scan logic, reader routes, notifications and push
templates/ (base, 7 pages, error + includes: nav, macros, add-item-modal)	HTML pages
static/css/style.css	Whole theme
static/js/ (13 files)	Page scripts, bell, toast, service worker
static/images/	Logo and favicon
database/	schema.sql, hardening.sql, iot.sql, tags_per_user.sql
firmware/rfid_scanner/rfid_scanner.ino, firmware/README.md	ESP32 sketch and wiring guide
tools/	fake_reader.py, pretend_worn.py, make_vapid_keys.py
tests/	run_iot_checks.py and fakedb.py
docs/	PROJECT_LOG, HANDOFF_FOR_REPORT, SECURITY, HOW_IT_WORKS, ARCHITECTURE, PRD, DESIGN, RULES, TASKS, THEME_PROMPT, PROJECT_MAP, the PDFs and report_assets (screenshots)

Environment variables (.env)

Name	Meaning
SUPABASE_URL	Address of the Supabase project (https)
SUPABASE_ANON_KEY	Public key (only checked, not used by the browser)
SUPABASE_SERVICE_ROLE_KEY	Full-power server key. Never reaches the browser
FLASK_SECRET_KEY	Signs login cookies, 32+ characters
VAPID_PUBLIC_KEY / VAPID_PRIVATE_KEY / VAPID_SUBJECT	Browser push signing keys (optional; push is off without them)
HOST	Listening address: 127.0.0.1 (default, this computer only) or 0.0.0.0 (so the ESP32 can reach it). The port is 5000
FLASK_ENV / FLASK_DEBUG / PROXY_HOPS	production switch / debug (never in production) / number of trusted proxies

15. Appendix B: glossary

Term	Meaning
RFID	Radio-frequency identification: a card or sticker that can be read without touching, using a radio field
RC522	A low-cost 13.56 MHz RFID reader chip on a small board that talks SPI
ESP32	A cheap microcontroller with built-in Wi-Fi and Bluetooth
SPI	A fast wire protocol with clock, MOSI, MISO and chip-select lines, used between ESP32 and RC522
Tag id / UID	The unique hexadecimal number stored in an RFID card, such as F3589104
Flask / blueprint	Python web framework / a way to split routes into feature files
Jinja	Flask's HTML template language with automatic escaping
Supabase	Hosted Postgres database with login and file storage
RLS	Row Level Security: database rules about who may read which rows
CSRF	Cross-site request forgery: a hostile page makes your browser send an action. Stopped with a secret token
XSS	Cross-site scripting: injecting script into a page. Stopped by escaping output and a strict CSP
CSP / nonce	Content-Security-Policy tells the browser which scripts may run; a nonce is a one-time code that marks our own inline script
Rate limiting	Allowing only so many requests per minute from one source
Hash	A one-way fingerprint of a secret (bcrypt for passwords, SHA-256 for device keys)
Web Push / VAPID	Standard way for a server to send notifications to a browser; VAPID keys prove the server's identity
Service worker	A script the browser runs in the background; here it shows push pop-ups
Toast	A small pop-up message inside the page
Debounce	Ignoring repeated signals that arrive within a short time
NVS / Preferences	The ESP32's permanent memory for small settings
SSRF	Server-side request forgery: tricking a server into contacting an address chosen by an attacker